

Dokumentacja: Implementacja Wielomianów na Bazie Słowników Pythona

Anna Dębska

04.02.2025

Spis treści

1 Wprowadzenie	3
2 Opis problemu	3
3 Opis Interfejsu	3
3.1 Tworzenie wielomianu	3
3.2 Wyświetlanie wielomianu	3
3.3 Odczyt stopnia wielomianu	4
3.4 Sprawdzanie, czy wielomian jest zerowy	4
3.5 Dodawanie wielomianów	4
3.6 Odejmowanie wielomianów	4
3.7 Mnożenie wielomianów	4
3.8 Obliczanie wartości wielomianu w punkcie	4
3.9 Porównywanie wielomianów	4
3.10 Dostęp do współczynnika przy danej potęgze	4
3.11 Całkowanie wielomianu.....	5
3.12 Różniczkowanie wielomianu	5
3.13 Interfejs użytkownika	5
4 Opis użytych struktur danych oraz rozwiązania	6
5 Analiza Złożoności Algorytmicznej	7
6 Podsumowanie i Wyniki Testów	7
7 Literatura	8

1 Wprowadzenie

Niniejszy dokument opisuje implementację **wielomianów** na bazie **słowników Pythona**. Klasa `Polynomial` umożliwia reprezentację i wykonywanie operacji na wielomianach, których współczynniki są przechowywane w formie słowników, gdzie klucze stanowią odpowiednie stopnie wielomianu. Projekt zawiera zaawansowane metody manipulacji wielomianami, funkcje wspierające testowanie poprawności działania kodu oraz interaktywny interfejs użytkownika.

2 Opis Problemu

Wielomiany (ang. *Polynomials*) są powszechnie stosowanymi obiektami matematycznymi w wielu dziedzinach, takich jak algebra, analiza matematyczna czy numeryczne przetwarzanie danych. Problemem jest efektywne zaimplementowanie operacji matematycznych, takich jak:

- Dodawanie, odejmowanie i mnożenie wielomianów
- Porównywanie wielomianów
- Obliczanie wartości wielomianu dla podanej zmiennej x
- Porównywanie wielomianów
- Reprezentacja wielomianu w przyjaznej formie tekstowej.
- Całkowanie i różniczkowanie wielomianów.

Wymagania obejmują również zachowanie niskiej złożoności czasowej i pamięciowej oraz łatwość testowania i rozszerzania implementacji.

3 Opis Interfejsu

3.1 Tworzenie wielomianu

Konstruktor klasy `Polynomial` umożliwia inicjalizację wielomianu na podstawie słowników:

```
p = Polynomial({0: 1, 1: 2, 2: 3})    #  $1 + 2x + 3x^2$ 
```

3.2 Wyświetlanie wielomianu

```
print(Polynomial({0: 3, 1: 0, 2: -1}))    # wyświetla:  $-x^2 + 3$ 
```

3.3 Odczyt stopnia wielomianu: `degree`

<code>p.degree()</code>	# np. dla {0: 5, 1: 3, 2: 7}: 2
-------------------------	---------------------------------

3.4 Sprawdzanie, czy wielomian jest zerowy: `is_zero`

Metoda zwraca True, jeśli wszystkie współczynniki wielomianu są zerami

<code>p.is_zero()</code>	# np. dla {0: 0, 0: 0, 0: 0}: True
--------------------------	------------------------------------

3.5 Dodawanie wielomianów +

<code>p1 + p2</code>	# dodaje współczynniki odpowiadających sobie potęg
----------------------	--

3.6 Odejmowanie wielomianów -

<code>p1 - p2</code>	# odejmuje współczynniki odpowiadających sobie potęg
----------------------	--

3.7 Mnożenie wielomianów *

<code>p1 * p2</code>	# wynik to nowy wielomian o stopniu równym sumie stopni p1 i p2
----------------------	---

3.8 Obliczanie wartości wielomianu w punkcie: `__call__`

<code>p(2)</code>	# oblicza wartość wielomianu dla x = 2
-------------------	--

3.9 Porównywanie wielomianów `==` / `!=`

Operator `==` sprawdza, czy dwa wielomiany są równe. Wielomiany są uznawane za równe, jeśli ich różnica daje wielomian zerowy. Operator `!=` sprawdza, czy wielomiany nie są równe.

<code>p1 == p2</code>	# True, jeśli p1 i p2 są równe
<code>p1 != p2</code>	# True, jeśli p1 i p2 nie są równe

3.10 Dostęp do współczynnika przy danej potędze `[]`

<code>p[2]</code>	# zwraca współczynnik przy x^2
<code>p[100]</code>	# zwraca 0, jeśli współczynnik nie istnieje

3.11 Całkowanie wielomianu `integrate`

```
p.integrate()    # np. dla 3x^2 + 2x + 1: x^3 + x^2 + x
```

3.12 Różniczkowanie wielomianu `differentiate`

```
p.differentiate()    # np. dla 3x^2 + 2x + 1: 6x + 2
```

3.13 Interfejs użytkownika

Do programu został dodany interfejs użytkownika, który umożliwia użytkownikowi wykonywanie operacji na wielomianach w sposób interaktywny. Zawiera on menu, które prowadzi użytkownika przez dostępne opcje:

```
Menu:
1. Dodaj wielomian
2. Odejmij wielomian
3. Pomnóż wielomian
4. Oblicz wartość wielomianu dla x
5. Wyświetl wielomian
6. Porównaj wielomiany
7. Odczytaj współczynnik przy potęgce x
8. Oblicz całkę z wielomianu
9. Zróżniczkuj wielomian
0. Wyjdź
```

Menu interfejsu użytkownika

Wprowadzanie wielomianów odbywa się poprzez podanie słownika współczynników, a wyniki są wyświetlane w czytelnej formie.

Przykład działania programu:

```
Wybierz opcję: 1
Dodawanie wielomianów
Podaj współczynniki wielomianu oddzielone spacjami (od najniższego stopnia):
Podaj pierwszy wielomian: 0:1 1:5 2:7
Podaj drugi wielomian: 0:2 1:8 2:4 3:9 4:5 5:0
Wynik: (7.0x^2 + 5.0x + 1.0) + (5.0x^4 + 9.0x^3 + 4.0x^2 + 8.0x + 2.0) = 5.0x^4 + 9.0x^3 + 11.0x^2 + 13.0x + 3.0
```

Przykład dodawania dwóch wielomianów (wartości oznaczone na zielono są wartościami podawanymi przez użytkownika)

Interfejs programu można uruchomić za pomocą polecenia:

```
python main.py
```

Dane wprowadza się w formacie:

```
Stopień_jednomianu:wartość_współczynnika    np. 0:3 2:-5 4:2
```

4 Opis użytych struktur danych oraz rozwiązania

Implementacja opiera się na wykorzystaniu słowników Pythona w celu przechowywania współczynników wielomianu, gdzie kluczem jest stopień jednomianu, a wartością jego współczynnik. Zastosowanie tej struktury danych pozwala uniknąć przechowywania zbędnych zer w przypadku wielomianów rzadkich co znacząco poprawia efektywność pamięciową i czasową operacji.

Słowniki w Pythonie umożliwiają szybki dostęp do elementów w czasie $O(1)$ oraz dynamiczne zarządzanie pamięcią.

Podstawowe operacje w pseudokodzie:

- Reprezentacja wielomianu w słowniku:

$P = \{0: 3, 2: -5, 4: 2\}$ # Reprezentuje $2x^4 - 5x^2 + 3$

- Dodawanie wielomianów

FUNKCJA DODAJ(P1, P2):

DLA każdej potęgi k w P1 i P2:

$wynik[k] = P1[k] + P2[k]$

ZWRÓĆ wynik

- Odejmowanie wielomianów

FUNKCJA ODEJMIJ(P1, P2):

DLA każdej potęgi k w P1 i P2:

$wynik[k] = P1[k] - P2[k]$

ZWRÓĆ wynik

- Mnożenie wielomianów

FUNKCJA MNOŻ(P1, P2):

$wynik = \{\}$

DLA każdej potęgi $k1$ w P1:

 DLA każdej potęgi $k2$ w P2:

```
wynik[k1 + k2] += P1[k1] * P2[k2]
ZWRÓĆ wynik
```

- Obliczanie wartości wielomianu (Algorytm Hornera)

FUNKCJA OBLICZ(P, x):

```
wynik = 0
```

```
DLA każdej potęgi k (od największej do najmniejszej):
```

```
    wynik = wynik * x + P[k]
```

```
ZWRÓĆ wynik
```

5 Analiza Złożoności Algorytmicznej

Struktura danych – słownik

Operacja	Złożoność czasowa	Złożoność pamięciowa
Inicjalizacja <code>__init__</code>	$O(n)$	$O(n)$
Odczyt stopnia <code>degree</code>	$O(1)$	$O(1)$
Czy zerowy <code>is_zero</code>	$O(1)$	$O(1)$
Odczyt współczynnika <code>__getitem__</code>	$O(1)$	$O(1)$

Główne algorytmy

Algorytm	Złożoność czasowa	Złożoność pamięciowa
Dodawanie <code>__add__</code>	$O(n)$	$O(n)$
Odejmowanie <code>__sub__</code>	$O(n)$	$O(n)$
Mnożenie <code>__mul__</code>	$O(n*m)$	$O(n+m)$
Obliczanie wartości(Horner) <code>__call__</code>	$O(n)$	$O(1)$
Całkowanie <code>integrate</code>	$O(n)$	$O(n)$
Różniczkowanie <code>differentiate</code>	$O(n)$	$O(n)$

Gdzie n i m to stopnie wielomianów

6 Podsumowanie i Wyniki Testów

Projekt oferuje efektywne i przejrzyste narzędzie do pracy z wielomianami, zapewniając wydajność operacji, czytelność kodu i łatwość użytkowania. Implementacja uwzględnia najważniejsze operacje algebraiczne oraz optymalizacje, takie jak algorytm Hornera. Dzięki intuicyjnemu interfejsowi użytkownika można w łatwy sposób wykonywać wybrane operacje, natomiast dzięki obecności testów możliwe jest bezproblemowe rozszerzanie funkcjonalności.

Projekt zawiera moduł `unittest` do testowania funkcjonalności klasy `Polynomial`. W pliku `test.py` zawarto następujące testy:

- Sprawdzanie poprawności interpretowania danych wejściowych
- Sprawdzenie operacji dodawania, odejmowania i mnożenia
- Sprawdzenie operatorów `==` i `!=`
- Sprawdzenie poprawności wyświetlania wielomianów
- Sprawdzenie poprawności odczytu współczynników i stopnia wielomianu
- Sprawdzenie poprawności wartościowania (algorytmu Hornera)
- Sprawdzanie poprawności całkowania
- Sprawdzanie poprawności różniczkowania

Testy można uruchomić za pomocą polecenia:

```
python test.py
```

Wszystkie testy zakończyły się pomyślnie, co potwierdza poprawność i niezawodność implementacji.

7 Literatura

- "Introduction to Algorithms" - T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein.
- Python Documentation: <https://docs.python.org/3/>.
- Kursy akademickie z algebry